

Technical Reference Manual

MagicBox v5.3 — hardware, firmware, data flow

Firmware • Sensors • Data Flow • Games • Exercises

Physics formulas and software implementation

Firmware version: **1.3.4** (USB only) / **2.2.5** (USB + Wi-Fi)
Application version: **v5.3**

Last updated: **September 2026**

Contents

1	Overview	2
1.1	System architecture	2
1.2	Main components	2
2	Hardware and sensors	3
2.1	Microcontroller: ESP32	3
2.2	IMU sensor: MPU-6500	3
2.2.1	Specifications	3
2.2.2	LSB-to-physical-units conversion	4
2.2.3	The on-chip low-pass filter (DLPF)	4
2.2.4	Mounting correction (180° rotation about the Y axis)	4
2.2.5	Computing angles from the accelerometer	5
2.2.6	Data emitted (JSON)	5
2.3	Weather sensor: BME280	5
2.3.1	Specifications	6
2.3.2	The barometric formula (altitude from pressure)	6
2.3.3	Dew point (Magnus formula)	7
2.3.4	Data emitted (JSON)	7
2.4	Light sensor: VEML7700	7
2.4.1	Specifications	7
2.4.2	Parameters emitted	8
2.4.3	Interpretation thresholds	8
2.5	CO ₂ sensor: SCD41	8
2.5.1	Specifications	8
2.5.2	Automatic self-calibration (ASC)	9
2.5.3	Thresholds used by the application (a ventilation indicator)	9
2.5.4	CO ₂ rate of change	10
2.5.5	Data emitted (JSON)	10
3	Firmware	11
3.1	Version history	11
3.2	The serial protocol	11
3.2.1	Boot sequence (once)	11
3.2.2	Main loop — timing	12
3.3	The device ID	12
3.4	I ² C bus self-healing — the 9-clock method	12
3.5	Wi-Fi: setup, discovery, reconnection	12
4	Data flow in the application	14
4.1	SerialManager	14
4.1.1	Inferring the type from JSON keys	14

4.1.2	Firmware version	14
4.2	DataManager	14
4.3	Full data path	15
4.4	Logs, exports and settings (v5.3)	15
4.4.1	Logs on disk	15
4.4.2	Exports	15
4.4.3	Relevant settings	15
5	Educational games	16
5.1	Among the Clouds — a flying game	16
5.1.1	Principle	16
5.1.2	Control formulas	16
5.1.3	Collision detection	16
5.1.4	Game parameters	17
5.2	Earthquake! — evacuating a building	17
5.2.1	Physical principle	17
5.2.2	Shock intensity from magnitude	17
5.2.3	Mobility reduction during the shock	17
5.2.4	Shelters and scores	18
5.2.5	Jump detection (EMA filter on a_z)	18
5.2.6	Aftershock magnitude	18
5.3	Resonance — damped harmonic oscillators	18
5.3.1	The physical model	18
5.3.2	Fair amplitude normalization	19
5.3.3	Shake frequency (from zero crossings)	19
5.3.4	Building parameters	19
5.4	Gyro Target — neuroplasticity	19
5.4.1	The target's Lissajous trajectory (levels 2 and 4)	19
5.4.2	Reversed controls (levels 3 and 4)	20
6	Guided exercises	21
6.1	The Seismometer	21
6.1.1	Vibration filter (high-pass on each axis)	21
6.1.2	The shaking index	21
6.1.3	Chart scale and status thresholds	21
6.2	CO ₂ and Photosynthesis	22
6.2.1	Chemical context	22
6.2.2	Independent rates of change	22
6.2.3	Net and gross photosynthesis	22
6.2.4	Molar CO ₂ flux (advanced calculation)	22
6.3	Pressure and Altitude	23
7	Explore apps	24
7.1	Air Quality — several boxes	24
7.2	Heat Islands with Several Boxes	24
7.3	Data export (PNG and Excel)	24
8	Windows: installation and troubleshooting	25
8.1	Installing the package	25
8.2	USB and Wi-Fi connection	25
8.3	Common problems	26

A Quick reference — JSON formats	27
A.1 Multi-sensor packet (1 Hz)	27
A.2 IMU packet (50 Hz)	27
A.3 SCD41 + BME280 packet	27
B I²C wiring	28

Chapter 1

Overview

MagicBox units — known in Romanian as *Cutiutele Magice* — are educational devices built around the **ESP32** microcontroller, fitted with four physical sensors. Each box streams data in real time to a desktop application (PyQt6 + matplotlib), where students can visualise, analyse and export it.

The application starts **in Romanian**; English is chosen in **Settings** (*Setări*) → **Language** (the change applies after a restart). Internal identifiers (sensor keys, module names, the Wi-Fi discovery token) are not translated and are the same in both languages.

1.1 System architecture

Layer	Component	Role
Hardware	ESP32 + sensors	Physical data acquisition
Firmware	<code>serial_wifi</code> 2.2.5, <code>serial_only</code> 1.3.4	Sensor reading, JSON
Transport	USB-Serial 115200 baud; Wi-Fi (TCP)	Data transmission
Reception	<code>SerialManager</code> , <code>WifiManager</code>	One thread per link
Storage	<code>DataManager</code>	Circular buffer + CSV logs on disk
Display	PyQt6 + matplotlib	Games, exercises, apps

1.2 Main components

- **4 sensors**: MPU-6500 (accelerometer + gyroscope), BME280 (temperature, humidity, pressure), VEML7700 (light), SCD41 (CO₂, optional)
- **4 games**: Among the Clouds, Earthquake!, Resonance, Gyro Target
- **15 guided exercises** in physics, chemistry, biology and geography
- **8 explore apps** (Air Quality, Heat Islands with Several Boxes, Weather Station, Solar Energy, Spectroscopy, The Greenhouse Effect, Seismology, Measurement Notebook)

Chapter 2

Hardware and sensors

2.1 Microcontroller: ESP32

Parameter	Value
CPU	Xtensa LX6 dual-core, 240 MHz
RAM	520 KB SRAM
Flash	4 MB
Interfaces	I ² C, SPI, UART, Wi-Fi, BLE
I ² C (firmware)	SDA=GPIO21, SCL=GPIO22, 100 kHz
Power	3.3 V
USB-serial adapter	CP210x or CH340 (depends on the board)

Why 100 kHz? The I²C bus runs at 100 kHz because the SCD41 sensor supports a maximum of 100 kHz per the Sensirion datasheet. At 400 kHz, the SCD41 corrupted the other sensors' transactions through *clock stretching*, and the MPU-6500 disappeared from the bus.

The box warms itself. The ESP32 dissipates around half a watt (more with Wi-Fi active), and the sensors share its enclosure. The BME280 therefore reads the air inside the box, not the room air: after a few minutes of running, the temperature can read one to a few degrees above the room, and the relative humidity correspondingly lower. To compare boxes, first place them side by side for a few minutes and note how they differ; for absolute values, let the box settle and compare it with a reference thermometer.

2.2 IMU sensor: MPU-6500

2.2.1 Specifications

Parameter	Configured value	Notes
Accelerometer	$\pm 8 g$	4096 LSB/ g
Gyroscope	$\pm 500^\circ/\text{s}$	65.5 LSB/ $(^\circ/\text{s})$
Magnetometer	—	the MPU-6500 has no magnetometer
Low-pass filter (DLPF)	$\approx 20 \text{ Hz}$	from firmware 2.2.5 / 1.3.4
Zero offset	$\approx \pm 0.06 g$	typical, per axis (datasheet)
I ² C address	0x68	—
Output rate	50 Hz	$\Delta t = 20 \text{ ms}$

The zero offset is not calibrated in the firmware. In practice a box lying flat may show a few hundredths of a g on a_x and a_y (an apparent angle of $1 \dots 3^\circ$) — which is why the *Flying Straight* exercise uses the 180° reversal test: half the sum of the two readings is the offset, half the difference is the real tilt of the table.

2.2.2 LSB-to-physical-units conversion

The firmware reads 14 consecutive bytes starting at register 0x3B (3 accelerometer axes + internal temperature + 3 gyroscope axes):

Accelerometer and gyroscope conversion

$$a_x = s_{ax} \cdot \frac{\text{raw}_{ax}}{4096} \quad [g] \quad (2.1)$$

$$\omega_x = s_{gx} \cdot \frac{\text{raw}_{gx}}{65.5} \quad [^\circ/\text{s}] \quad (2.2)$$

where $s_{ax}, s_{gx} \in \{-1, +1\}$ are the mounting-correction signs.

Firmware implementation (C):

```
out[0] = IMU_SIGN_AX * (float)((int16_t)(b[0]<<8|b[1])) / 4096.0f; // ax
out[3] = IMU_SIGN_GX * (float)((int16_t)(b[8]<<8|b[9])) / 65.5f; // gx
```

2.2.3 The on-chip low-pass filter (DLPF)

The box sends 50 samples per second, so it can faithfully reproduce only signals below the 25 Hz Nyquist frequency. Out of reset, unconfigured, the MPU-6500 passes up to 250 Hz (gyroscope) and 460 Hz (accelerometer): the vibration of a fan or a motor then folds down into fake slow signals (*aliasing*). From firmware **2.2.5** (Wi-Fi) and **1.3.4** (USB only), the firmware writes at every initialisation:

```
mpuWrite(0x1A, 0x04); // CONFIG:      gyro DLPF_CFG=4 -> ~20 Hz, ~10 ms delay
mpuWrite(0x1D, 0x04); // ACCEL_CONFIG2: accel A_DLPF_CFG=4 -> ~21 Hz, ~12 ms delay
```

Hand movements, the oscillations in *Resonance* ($1 \dots 3.3$ Hz) and the taps in *The Seismometer* pass practically unattenuated; components above ≈ 20 Hz are attenuated instead of showing up as fake slow motion. The filter is re-applied after an automatic IMU re-initialisation too.

2.2.4 Mounting correction (180° rotation about the Y axis)

The MPU-6500 board is mounted face down inside the box. The firmware corrects this mounting with a **180° rotation about the Y axis** — never by mirroring the Z axis alone, which would invert the reference frame's orientation instead of restoring it to an upright one. A 180° rotation about Y leaves the Y axis unchanged and requires negating the other **two** axes (for the accelerometer and the gyroscope alike):

Axis	Accelerometer sign	Gyroscope sign
X	−1	−1
Y	+1	+1
Z	−1	−1

As a result, a box lying flat (on its bottom) reports $a_z \approx +1g$: the table pushes the box up, and the accelerometer measures that support force (in free fall it would read 0). **A box that reports $a_z \approx -1g$ while lying flat is running firmware older than v2.2.1 (the Wi-Fi sketch, serial_wifi) or v1.3.0 (the USB-only sketch, serial_only) and must be reflashed** — the wrong sign is a firmware defect, not something to compensate for in the application. The v5.3 application shows this warning itself (see Ch. 4).

A box whose sensor is mounted the other way. If a box shows $a_z \approx -1g$ on the table although its firmware is current (2.2.1 / 1.3.0 or newer), its MPU-6500 board faces up, the opposite of the boards the firmware correction is written for. No other firmware is needed: in **Settings** → **Box names** tick **Motion sensor upside down** for that box. The application then applies the same 180° rotation about Y (negating X and Z, for the accelerometer and the gyroscope) before it shows, saves or uses the data in activities. The tick is remembered by the box's ID (BOX_XXXX).

Check rule: tilt the right edge down $\Rightarrow$ roll and the gyroscope must have the same sign.

2.2.5 Computing angles from the accelerometer

Roll and pitch (from static acceleration)

$$\phi_{\text{roll}} = \arctan\left(\frac{a_y}{\sqrt{a_x^2 + a_z^2}}\right) \cdot \frac{180^\circ}{\pi} \quad (2.3)$$

$$\theta_{\text{pitch}} = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \cdot \frac{180^\circ}{\pi} \quad (2.4)$$

Input: a_x, a_y, a_z in g ; output in degrees ($^\circ$).

Full roll (the aircraft picture over $\pm 180^\circ$, including upside-down flight):

$$\phi_{\text{full}} = \text{atan2}(a_y, a_z) \cdot \frac{180^\circ}{\pi}$$

2.2.6 Data emitted (JSON)

IMU packet (50 Hz):

```
{ "id": "BOX_A1B2", "ts_ms": 12345, "imu_hz": 50, "imu":
  ":[[-0.023, 0.014, 0.998, 0.152, -0.089, 0.041]]"
```

Field order: $[a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]$, accelerations in g , angular rates in $^\circ/\text{s}$.

2.3 Weather sensor: BME280

2.3.1 Specifications

Quantity	Range	Resolution	Accuracy (datasheet)	Measured noise
Temperature	−40... + 85 °C	0.01 °C	±1 °C	0.003 °C
Humidity	0...100 %	0.01 %	±3 % RH	0.1 % RH
Pressure	300...1100 hPa	0.18 Pa	±1 hPa absolute ±0.12 hPa relative	1.4 Pa

I²C address 0x76 or 0x77; normal mode, ×1 oversampling, ×2 IIR filter, one reading per second. *Measured noise* is the standard deviation of successive readings from a box at rest: it shows how small a *change* the box can see. *Accuracy* shows how far the absolute value can be from the true one. The difference matters:

- Pressure: a difference of 0.12 hPa (one metre of altitude) is clearly visible, even though the absolute value may be off by up to 1 hPa. The 1.4 Pa noise corresponds to about 12 cm of altitude.
- Temperature: the box sees changes of hundredths of a degree, but two boxes can differ by a degree or more — partly because of self-heating (see the ESP32 note).

The firmware sends values with **2 decimal places**: with a single decimal, the chart showed 0.1 °C steps and the real resolution was wasted. The application *displays* pressure with one decimal (two at the **Scientist** level) and altitude with one decimal; data files keep every decimal received.

2.3.2 The barometric formula (altitude from pressure)

International barometric formula (standard atmosphere)

$$h = \frac{T_0}{L} \left[1 - \left(\frac{P}{P_0} \right)^{\frac{R^* L}{g_0 M}} \right] = 44\,330 \cdot \left[1 - \left(\frac{P}{P_0} \right)^{0.1903} \right] \quad (2.5)$$

- h = altitude [m]; P = measured pressure [hPa]
- P_0 = sea-level pressure (default 1013.25 hPa; the QNH value is entered in **Settings** and kept)
- $T_0 = 288.15$ K, $L = 0.0065$ K/m (the temperature lapse rate), so $T_0/L \approx 44\,330$ m
- exponent $\frac{R^* L}{g_0 M} = \frac{8.3145 \times 0.0065}{9.80665 \times 0.028964} \approx 0.1903 \approx \frac{1}{5.255}$, with R^* the universal gas constant and M the molar mass of dry air

Practical sensitivity: $\Delta P \approx 0.12$ hPa per metre (≈ 8.3 m/hPa) near sea level. One floor (≈ 3 m) produces $\Delta P \approx 0.35$ hPa.

With $P_0 = 1013.25$ hPa, the altitude shown is not the altitude of the place: atmospheric pressure changes from day to day by tens of hPa, i.e. hundreds of metres of apparent altitude. To compare with your town's altitude, enter the day's QNH (from an airport or a nearby weather station) in **Settings**; even then an uncertainty of the order of ± 10 m remains from the sensor's absolute accuracy.

2.3.3 Dew point (Magnus formula)

Dew point — the Magnus formula

$$\gamma(T, H) = \frac{aT}{b + T} + \ln\left(\frac{H}{100}\right) \quad (2.6)$$

$$T_d = \frac{b\gamma}{a - \gamma} \quad (2.7)$$

Constants (Alduchov–Eskridge): $a = 17.625$, $b = 243.04$ °C; input T in °C, H in %. Every screen of the application uses the same constants.

2.3.4 Data emitted (JSON)

Slow-sensor packet (1 Hz):

```
{"id":"BOX_A1B2","ts_ms":45123,"data":{"lux":312.5,"white":1840,"raw_als":1821,
  "T":22.34,"H":58.12,"P":1013.28,"ALT":0.3}}
```

2.4 Light sensor: VEML7700

2.4.1 Specifications

Parameter	Firmware $\leq$ 2.2.4 / 1.3.3	Firmware $\geq$ 2.2.5 / 1.3.4
Useful range	$0 \dots \approx 3\,800$ lx (saturates)	$\approx 0.004 \dots \approx 120\,000$ lx
Gain / integration time	fixed: $\times 1$, 100 ms	automatic: $\times \frac{1}{8} \dots \times 2$, 25 $\dots$ 800 ms
Resolution	0.0576 lx	0.0036 $\dots$ 1.84 lx (per rung)
Non-linearity correction	no	yes, within its valid range
Accuracy	$\approx \pm 10\%$ below ≈ 10 klx; $\pm 15 \dots 20\%$ above (only $\geq$ 2.2.5 / 1.3.4)	
I ² C address	0x10 (fixed)	
Output rate	1 Hz (the last valid reading is sent)	

Up to firmware 2.2.4 / 1.3.3, the sensor ran at a fixed gain of $\times 1$ and 100 ms integration. Its 16-bit counter filled up at $65\,535 \times 0.0576 \approx 3\,775$ lx: *any* brighter light — near a sunny window, outdoors in the shade, in the sun — showed as $\approx 3\,800$ lx. A box that reads $\approx 3\,800$ lx in full sun must be reflashed.

From firmware 2.2.5 / 1.3.4, the firmware chooses gain and integration time itself from a 9-rung ladder (from $\times \frac{1}{8}/25$ ms for direct sun to $\times 2/800$ ms for near darkness). It steps to a less sensitive rung when the count exceeds 50 000 (two rungs at once when saturated) and to a more sensitive one when it drops below 1 000. The switch never pauses the main loop (a state machine that only compares `millis()`), so the 50 Hz IMU stream is not interrupted. The least sensitive rung (1.8432 lx/count) reaches $65\,535 \times 1.8432 \approx 120\,800$ lx, i.e. direct summer sun. After a rung change the first new value arrives after about two integrations; until then the last valid reading is re-sent.

Absolute values in bright light. Above $\approx 10\,000$ lx, absolute values are approximate ($\pm 15 \dots 20\%$). The activities rely on relative comparisons — the panel's tilt series, sun versus shade, the same box in the same place —, which remain sound.

2.4.2 Parameters emitted

LUX [lx] — illuminance computed from the ALS channel. From 2.2.5 / 1.3.4: $\text{count} \times$ the current rung's resolution, with the Vishay non-linearity correction (application note 84323, the same one the Adafruit library's automatic mode applies), used only up to 10 000 lx, where its polynomial is valid; above that the value continues linearly, joined without a jump (capped at 150 000 lx). Up to 2.2.4 / 1.3.3: $\text{count} \times 0.0576$, without correction — the value was *not* “ALS-compensated”.

WHITE — the white channel. **RAW** — the ALS channel. From 2.2.5 / 1.3.4 both are reported *on the old scale* (the equivalent at gain $\times 1 / 100$ ms), so old and new files stay comparable; that is why they can exceed 65 535 in bright light.

2.4.3 Interpretation thresholds

Range [lx]	Description
< 1	Darkness
1...50	Low light (night / dimly lit indoors)
50...500	Ordinary indoor lighting
500...2 000	Overcast day / near a window
2 000...20 000	Indirect daylight, outdoors in the shade
20 000...80 000	Direct sun (partial, winter, morning)
> 80 000	Strong direct sun

The last three rows can only be reached with firmware 2.2.5 / 1.3.4 or newer; with older firmware, everything above $\approx 3\,800$ lx falls into the same row.

2.5 CO₂ sensor: SCD41

2.5.1 Specifications

Parameter	Value	
Specified range	400...5 000 ppm	may also report below 400 ppm
Accuracy	$\pm(40 \text{ ppm} + 5\% \text{ of reading})$	within the specified range
Principle	photoacoustic NDIR (infrared)	
Settling	≈ 30 s	first readings may be unstable
Measurement cycle	≈ 5 s	set by the sensor, not the firmware
Self-calibration (ASC)	on	see below
I ² C address	0x62	fixed (Sensirion)

The SCD41 completes a measurement every ≈ 5 s, slower than the other sensors (1 Hz). The firmware keeps the last value read and repeats it in every 1 Hz packet so the data stream stays regular — the CO₂ values do not, in fact, update on every packet. From v5.3 the application keeps a time axis of its own for CO₂ (see Ch. 4), so charts no longer stretch or squeeze the curve.

Which firmware reads the SCD41. The Wi-Fi sketch (`serial_wifi`) reads it from 2.2.0, and the USB-only sketch (`serial_only`) from **1.3.4**, with the same JSON keys and the same 5 s cycle. A `serial_only` older than 1.3.4 ignores the sensor even when it is fitted: CO₂ activities then need the box to be reflashed. If the SCD41 is absent from the bus, both firmwares ignore it and send the rest of the data normally.

2.5.2 Automatic self-calibration (ASC)

An NDIR sensor slowly loses its reference over time. The SCD41 has *Automatic Self-Calibration* (ASC): it assumes that, over a period of operation (about 44 hours at first, then about 156 hours, i.e. a week), the lowest concentration it has seen is fresh outdoor air, ≈ 400 ppm, and slowly shifts its reference accordingly.

- **How the firmware sets it.** Both variants of the firmware (`serial_wifi` 2.2.x and `serial_only` 1.3.4) start the sensor with `scd41.begin(Wire)` from the SparkFun SCD4x library. That function enables ASC at *every boot* (default argument `autoCalibrate = true`). The firmware has no other calibration command.
- **When it helps.** A classroom aired well at least once a week (or a box taken outdoors now and then) keeps its calibration by itself.
- **When it hurts.** A box that never sees fresh air (a cupboard, an unventilated room) will “learn” a higher value as 400 ppm and read too low. Conversely, a sealed jar with a plant in the light, in which CO₂ stays below 400 ppm for hours, can push the reference up. For long experiments below 400 ppm, disable ASC.

Disabling ASC. Because the firmware re-enables ASC at every boot, it is disabled in the box’s sketch (Wi-Fi or USB-only), by changing a single line and reflashing the box:

```
scd41_ok = scd41.begin(Wire, true, false); // measBegin=true, autoCalibrate=false
```

Forced recalibration (FRC) outdoors. If a box reads visibly wrong in clean air, it can be recalibrated against a known concentration:

1. Take the box outdoors, in the shade, away from people, cars and chimneys, and let it measure for at least 3 minutes (outdoor air is about 420 ppm today).
2. With a short sketch (or calls added temporarily to the firmware), stop periodic measurement and request the recalibration; the function returns the correction applied, in ppm:

```
scd41.stopPeriodicMeasurement(); // waits 500 ms
float corr = scd41.performForcedRecalibration(420);
```

3. Restart periodic measurement (`startPeriodicMeasurement()`) or restart the box, and check that it reads around 420 ppm outdoors.

The current firmware exposes no calibration commands on the serial port; both operations therefore require reflashing the box (see `FLASHING_GUIDE.pdf` or `FLASHING_GUIDE_R0.pdf` in the `firmware` folder). Do not run an FRC in the classroom: the concentration there is not known.

2.5.3 Thresholds used by the application (a ventilation indicator)

At classroom levels, CO₂ is not a poison: it is an *indicator of ventilation* — how much of the air people have breathed out is still in the room. The WHO has no guideline value for indoor CO₂; the thresholds below are practical thresholds used by the application, close to the indoor air quality categories of standard **EN 16798-1** (which expresses them as the excess over outdoor air: roughly +550, +800 and +1350 ppm for categories I, II and III).

CO ₂ [ppm]	Quality	Action
< 800	Very good	—
800 ... 1200	Good	—
1200 ... 2000	Moderate	Ventilation recommended
2000 ... 5000	Poor	Ventilate now
> 5000	Above the occupational limit	Ventilate; the workplace exposure limit is 5000 ppm (8-h average)

2.5.4 CO₂ rate of change

Rate of change (the CO₂ exercise)

$$\dot{C} = \frac{\Delta \text{CO}_2}{\Delta t} = \frac{C_2 - C_1}{(t_2 - t_1)/60} \quad [\text{ppm}/\text{min}] \quad (2.8)$$

The quick indicator is recalculated every 30 s. For a rate you report, use the slope of a regression line over at least 3–5 minutes of data (two readings 30 s apart differ by noise and by the sensor's 5 s cycle). The arrows use neutral colours: a falling CO₂ is not “bad”, and a rising one is not “good”.

2.5.5 Data emitted (JSON)

```
{"id":"BOX_C3D4","ts_ms":5000,"data":{"co2":842,"TEMP_SCD":23.1,"HUM_SCD":55.4}}
```

Chapter 3

Firmware

3.1 Version history

There are two firmware variants whose versions advance in step: **serial_wifi** (2.x: USB + Wi-Fi, with CO₂) and **serial_only** (1.x: USB only; CO₂ from 1.3.4).

Wi-Fi	USB only	Main changes
2.0.0	1.2.0	Dual USB + Wi-Fi transport; IMU mounting correction
2.1.0	—	Wi-Fi setup portal (access point), no passwords in the code
2.2.0	—	SCD41 support (CO ₂ + T/RH), 5 s cycle
2.2.1	1.3.0	T/RH/P with 2 decimals; mounting correction as a rotation about Y (minimum supported version : below it, $a_z \approx -1\text{ g}$)
2.2.2	1.3.1	I ² C bus at 100 kHz
2.2.3	1.3.2	IMU self-healing: boot retries + bus reset
2.2.4	1.3.3	Bus unlock with 9 clock pulses; BME/VEML retry at boot; gives up after 30 attempts
2.2.5	1.3.4	Current. VEML7700 auto-range (up to $\approx 120\text{ klx}$); IMU low-pass filter $\approx 20\text{ Hz}$; (USB only) SCD41 support; (Wi-Fi) non-blocking TCP connect, with progressively spaced retries

3.2 The serial protocol

Baud rate: 115200, 8N1. The same lines are sent over Wi-Fi.

3.2.1 Boot sequence (once)

```
[BOOT] Cutiutele Magice fw v2.2.5 (build Sep 23 2026)
[VEML7700] OK
[BME280] OK
[MPU6500] OK
[SCD41] OK
DEVICE: BOX_A1B2
TYPE: MULTI
SENSORS: LUX,WHITE,RAW,T,H,P,ALT,AX,AY,AZ,GX,GY,GZ,CO2,TEMP_SCD,HUM_SCD
FW: 2.2.5
```

The application parses **DEVICE:**, **TYPE:**, **SENSORS:** and **FW:**. Lines starting with **[** (diagnostics) are ignored. The header is re-sent over TCP on every new connection.

3.2.2 Main loop — timing

Event	Frequency	JSON key
Slow sensors (VEML7700, BME280; SCD41 held)	1 Hz	"data"
IMU (MPU-6500)	50 Hz	"imu"
SCD41 read	0.2 Hz	included in "data"

3.3 The device ID

Generating the ID from the MAC address (C)

```
snprintf(DEVICE_ID, sizeof(DEVICE_ID), "BOX_%04X",
         (uint16_t)(ESP.getEfuseMac() >> 32));
```

Bytes 4–5 of the 6-byte MAC (eFuse) $\Rightarrow$ the format **BOX_XXXX**.
 Example: MAC AA:BB:CC:DD:EE:FF $\Rightarrow$ **BOX_DDEE**.

In the application, the teacher can give each box a name (for example “Group 1”) in **Settings** $\rightarrow$ **Box names**; the name is kept and appears everywhere in place of the ID, together with a fixed colour per box. The ID remains in the data files.

3.4 I²C bus self-healing — the 9-clock method

The SCD41 can hold SDA low (*clock stretching*) if power is lost mid-measurement. `Wire.end()` alone does not release the bus.

Unlocking procedure (9 clock pulses)

1. Set SDA (GPIO 21) as an input with pull-up
2. Generate 9 manual pulses on SCL (GPIO 22): HIGH / LOW, 5 μ s each
3. Stop at the first cycle where SDA goes HIGH (the device has released the bus)
4. Generate a STOP condition: SDA low $\rightarrow$ high with SCL=HIGH
5. `Wire.end()` + `Wire.begin(21,22)` + `Wire.setClock(100000)`
6. Re-initialise the MPU-6500 (including the low-pass filter)
7. After 30 failed attempts: IMU disabled until the next boot (BME280 + SCD41 continue normally)

3.5 Wi-Fi: setup, discovery, reconnection

`serial_wifi` firmware (2.x) only.

- **Setup.** With no saved network, the box opens an open access point. Its name is **CutiaMagica-XXXX**, where XXXX are the last 4 characters of the box ID. Connect to it with a phone or laptop; the setup page opens by itself (otherwise: 192.168.4.1). Choose the classroom network, enter the password, save; the box restarts and connects by itself from then on. If the saved network does not answer within 20 s, the portal reopens.
- **Erasing the saved network.** Hold the BOOT button while powering the box.

- **Discovery.** Every 3 s the application sends a UDP *broadcast* message on port 8766, of the form `MAGICBOX:EDUGEOLAB:<ip>:<port>`. The box listens for it and opens a TCP connection to the computer on port **8765**. The `MAGICBOX:` prefix and the `EDUGEOLAB` token must match in the application and the firmware; they are never renamed.
- **Spaced-out reconnection.** If the computer's firewall blocks port 8765, the connection fails. From 2.2.5 the connection attempt no longer blocks the loop (until then each attempt stopped the IMU stream for about 1.5 s every 5 s), and the pause between attempts doubles: $5 \rightarrow 10 \rightarrow 20 \rightarrow 40 \rightarrow 60$ s (maximum). It resets on a successful connection or when the discovery message comes from a different computer.
- **Dual transport.** The box sends the same lines over USB and Wi-Fi; the application drops packets received twice by their timestamp.
- **Network limits.** School networks with client isolation (devices cannot see each other) or WPA2-Enterprise authentication do not work; the fallback is a phone hotspot, joined by both the boxes and the computer.

Chapter 4

Data flow in the application

4.1 SerialManager

SerialManager scans COM ports every 3 s and launches one **SerialWorker** (QThread) per port. Each worker:

1. Opens the port at 115200 baud
2. Reads ASCII lines, identifies the device (at most 4 s) or infers the type from the first JSON packet's keys (if the box was already running)
3. Emits the `identified(device_id, type, sensors, port)` signal
4. Continues emitting `data_ready(device_id, payload)`
5. No data for 8 s $\Rightarrow$ automatic reconnection

Over Wi-Fi, **WifiManager** does the same: it listens on TCP 8765 and sends the discovery messages.

4.1.1 Inferring the type from JSON keys

Keys present in "data"	Detected type
T, H, P, ALT	BME280
lux, white, raw_als	VEML7700
co2, TEMP_SCD, HUM_SCD	SCD41
ax, ay, az, gx, gy, gz	MPU6500
Combination	MULTI

4.1.2 Firmware version

The application reads the FW: `x.y.z` line. Below **2.2.1** (Wi-Fi) or **1.3.0** (USB only) it shows a warning that the box must be reflashed (the Z axis would be upside down); below the latest version (2.2.5 / 1.3.4) it shows only a note that a newer version exists (in which the light sensor reaches full sunlight).

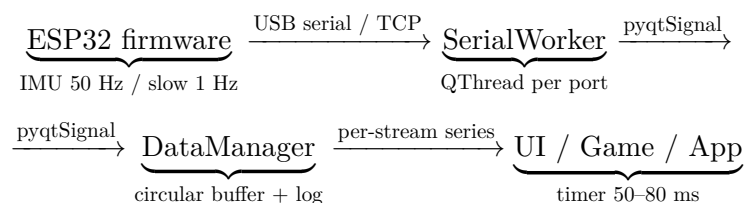
4.2 DataManager

Keeps a circular window per box in memory and writes everything to disk.

- **A separate time axis per stream.** The IMU (50 Hz), the slow sensors (1 Hz) and CO₂ (every 5 s) each keep their own timestamps. Up to v5.2 all streams shared a single list of times, and a 10-minute CO₂ curve could be drawn over 12 seconds.

- `get_latest(dev_id)` — the most recent point per sensor
- Time series are read per stream, with their own times; exercises and apps pick the box among those connected at that moment.
- Parsing: each handler (`MPU6500Handler`, `BME280Handler` etc.) maps the raw JSON keys to the standard keys (`AX`, `T`, `LUX` etc.)

4.3 Full data path



Typical USB-serial → chart latency: **50–100 ms** (plus ≈ 10 ms added by the IMU low-pass filter).

4.4 Logs, exports and settings (v5.3)

4.4.1 Logs on disk

- Each box has a CSV file for the slow sensors and an `_imu.csv` file for the IMU, in the `logs` folder next to the application.
- **Fallback folder.** If the application folder cannot be written to (a write-protected stick, a network share, **Program Files**), the application writes to `MagicBox\logs` in **Documents** and shows a warning. A write error (for example a file held open in Excel) stops only that box’s log, not the charts.
- **Time.** Each row carries an ISO timestamp with milliseconds, and a numeric `elapsed_s` column (seconds since the start of the recording), easy to use in spreadsheet charts. Example timestamp: `2026-09-23 10:15:02.345`.
- **Nothing is deleted automatically.** The **Keep logs for (days, 0 = forever)** setting defaults to 0; if the teacher picks a number of days, the application warns before deleting.
- Logs on disk use the “machine” format (comma separator, decimal point), whatever the language.

4.4.2 Exports

- Exports default to `.xlsx` (Excel), which opens correctly everywhere. A legend sheet describes each column, with the unit and the sensor’s accuracy.
- CSV export, when the application is in Romanian, uses `;` as the separator and the decimal comma, so Romanian Excel opens it in columns.
- The **Data & Export** page is visible at the **Scientist** and **Teacher** levels.

4.4.3 Relevant settings

Language (Romanian by default, English), **Projector mode** (larger charts and text), **Box names**, the QNH pressure for altitude (kept between sessions), **Keep logs for**; technical options (port speed, chart interval) are grouped under **Advanced**.

Chapter 5

Educational games

5.1 Among the Clouds — a flying game

5.1.1 Principle

The box is an aircraft's control stick: roll → left/right, pitch → up/down. Clouds are obstacles; rings score points. Turn the box over by 180° and the plane flies upside down.

5.1.2 Control formulas

Aircraft control from the accelerometer

$$\phi_{\text{roll}} = \arctan\left(\frac{a_y}{\sqrt{a_x^2 + a_z^2}}\right) \quad (5.1)$$

$$\theta_{\text{pitch}} = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \quad (5.2)$$

$$v_x = \text{clamp}\left(\frac{\phi}{30^\circ}, -1, 1\right) \times 260 \text{ px/s} \quad (5.3)$$

$$v_y = -\text{clamp}\left(\frac{\theta}{30^\circ}, -1, 1\right) \times 260 \text{ px/s} \quad (5.4)$$

Picture rotation (with upside-down flight): $\phi_{\text{full}} = \text{atan2}(a_y, a_z)$

5.1.3 Collision detection

$$d = \sqrt{(x_p - x_o)^2 + (y_p - y_o)^2} \quad (5.5)$$

- Cloud hit: $d < r_{\text{cloud}} + 16$
- Passing through a ring: $|x_p - x_o| < 12$ **and** $|y_p - y_o| < r_{\text{ring}} - 14$

5.1.4 Game parameters

Parameter	Value
Frames per second	20 (50 ms tick)
World speed (clouds)	150 px/s
Max aircraft speed	260 px/s
Angle for max speed	30°
Lives	3
Protection after a hit	1.5 s

5.2 Earthquake! — evacuating a building

5.2.1 Physical principle

The game simulates how a real earthquake unfolds and tackles the myths about taking shelter:

1. **P wave** (compressional, fast) — the early warning. For the intermediate-depth Vrancea earthquakes (60–200 km), the INFP warning system gives Bucharest about 25 s.
2. **S wave** (shear) — the main shaking; mobility drops sharply
3. **Aftershocks** — calm periods + smaller shocks

5.2.2 Shock intensity from magnitude

The magnitude in the game is a moment magnitude (M_w), not “on the Richter scale”.

Normalized intensity and time envelope

$$I(M) = \text{clamp}\left(\frac{M - 3.5}{3.0}, 0.15, 1.25\right) \quad (5.6)$$

$$\text{env}(t) = 0.45 + 0.55 \cdot \frac{t_{\text{remaining}}}{t_{\text{dur}}} \quad (5.7)$$

$$I_{\text{total}} = I(M) \cdot \text{env}(t) \quad (5.8)$$

Shock duration: $t_{\text{dur}} = 2.5 + (M - 4.0) \times 0.9$ [s]

5.2.3 Mobility reduction during the shock

$$\Delta x = v_{\text{player}} \cdot \underbrace{(1 - 0.75 \cdot \min(I_{\text{total}}, 1))}_{\text{hindrance factor}} \cdot \Delta t \quad (5.9)$$

At maximum intensity ($I = 1$), the player moves at 25% of normal speed — the realistic effect of a strong earthquake.

5.2.4 Shelters and scores

Shelter type	Radius [px]	Multiplier	Lesson
Under the table (masa)	48	$\times 2.0$	Full protection
Wall corner (colt)	30	$\times 1.5$	Good
Door frame (toc)	20	$\times 1.2$	Weak (a popular myth!)
No shelter	—	$\times 1.0$	Dangerous

5.2.5 Jump detection (EMA filter on a_z)

A jump = a dynamic spike in a_z

$$\tilde{a}_z[n] = \tilde{a}_z[n-1] + 0.08 \cdot (a_z[n] - \tilde{a}_z[n-1]) \quad (\text{slow EMA} = \text{gravity}) \quad (5.10)$$

$$\delta a_z[n] = a_z[n] - \tilde{a}_z[n] \quad (\text{dynamic component}) \quad (5.11)$$

$$\text{jump} = \delta a_z > \theta_{\text{jump}} \quad \theta_{\text{jump}} \in [0.20, 0.80] g \text{ (UI slider)} \quad (5.12)$$

5.2.6 Aftershock magnitude

$$M_{\text{aftershock}} = \min(M_0 - 0.8, M_0 - 2.0 + 0.18 \cdot n_{\text{aftershocks}} + \mathcal{U}(0, 0.5)) \quad (5.13)$$

with $M_0 = 5.8$ the mainshock magnitude. The $M_0 - 0.8$ ceiling follows Båth's law: the largest aftershock sits on average about one magnitude unit below the mainshock. (The ceiling used to be 7.0, so aftershocks could exceed the mainshock — unrealistic.)

5.3 Resonance — damped harmonic oscillators

5.3.1 The physical model

Each building is a **damped harmonic oscillator excited at its base** (a standard model in earthquake engineering):

Equation of motion with base excitation

$$\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2x = -a(t) \quad (5.14)$$

- x = the building's displacement relative to its base [m]
- $\omega_0 = 2\pi f_0$ = the natural angular frequency [rad/s]
- $\zeta = 0.05$ = the damping ratio (5%, typical for civil structures)
- $a(t)$ = the base acceleration (the hand) [m/s^2]

Integration by the **semi-implicit (symplectic) Euler** method at each IMU sample ($\Delta t \approx 20$ ms): the velocity is updated first, and the position uses the *new* velocity.

$$\dot{x}[n+1] = \dot{x}[n] + (-2\zeta\omega_0\dot{x}[n] - \omega_0^2x[n] - a[n])\Delta t \quad (5.15)$$

$$x[n+1] = x[n] + \dot{x}[n+1]\Delta t \quad (5.16)$$

Unlike the explicit Euler method (which would use $\dot{x}[n]$ in the second equation and add spurious energy at every step), the symplectic scheme conserves an undamped oscillator's energy on average and stays stable as long as $\omega_0\Delta t < 2$; for the 3.3 Hz house, $\omega_0\Delta t \approx 0.41$.

5.3.2 Fair amplitude normalization

Normalized amplitude (frequency-independent)

$$A_{\text{norm}} = |x| \cdot \frac{2\zeta}{X_{\text{ref}}} \quad (5.17)$$

$X_{\text{ref}} = 0.02 \text{ m}$ = the hand's reference displacement.

At resonance: $A_{\text{norm}} \approx 1$ for any building, given the same hand motion.

Why normalize on displacement, not acceleration?

The same hand motion ($\pm 2 \text{ cm}$) produces $(2\pi f)^2$ times more acceleration at high frequencies. Without normalization, the house (3.3 Hz) would dance at any quick tremor, while the tower (1 Hz) would seem impossible to set in motion.

5.3.3 Shake frequency (from zero crossings)

Frequency estimate from zero crossings

$$f_{\text{shake}} = \frac{N_{\text{crossings}}}{2 \cdot T_{\text{window}}} \quad (5.18)$$

Window: 1.5 s; signal threshold: $|a| > 0.4 \text{ m/s}^2$ (filters out noise).

5.3.4 Building parameters

Building	f_0 [Hz]	T_0 [s]	Floors	Rule of thumb
The Tower	1.0	1.0	10	$T \approx N_{\text{floors}}/10 \text{ s}$
The Apartment Block	2.0	0.5	5	
The House	3.3	0.3	3	

The buildings follow the rule of thumb $T \approx N/10 \text{ s}$. A building “dances” when its normalized amplitude passes the threshold *and* is clearly larger than the other two; hence the on-screen prompt: “Make each building dance on its own”.

Deformation shape (cantilever mode): floor k 's displacement $\propto (k/N_{\text{floors}})^2$.

5.4 Gyro Target — neuroplasticity

Four levels: (1) a fixed target at the centre; (2) a moving target; (3) reversed controls; (4) reversed + moving.

5.4.1 The target's Lissajous trajectory (levels 2 and 4)

Lissajous curve

$$x_{\text{target}}(t) = r \cdot \sin(\omega_a t + \delta) \quad (5.19)$$

$$y_{\text{target}}(t) = r \cdot \sin(\omega_b t) \quad (5.20)$$

The ratio ω_a/ω_b is randomized per round — a variable trajectory.

5.4.2 Reversed controls (levels 3 and 4)

Sensor negation: $a_y \rightarrow -a_y$, $a_x \rightarrow -a_x$. The brain has built “up = up” reflexes over years, and knowing the rule does not switch them off. Adaptation *begins* within minutes and shows in the falling times over successive attempts; how long a full switch takes depends on the person and the practice. (The app used to print a figure — “3–7 days of daily practice” — which was not a measured constant and has been removed.)

Chapter 6

Guided exercises

6.1 The Seismometer

6.1.1 Vibration filter (high-pass on each axis)

Extracting vibration: a separate baseline on each axis

$$b_k[n] = \alpha \cdot a_k[n] + (1 - \alpha) \cdot b_k[n - 1], \quad k \in \{x, y, z\} \quad (6.1)$$

$$(\alpha = 0.02 \text{ at } 50 \text{ Hz: slow EMA, } \tau \approx 1 \text{ s, } f_c \approx 0.16 \text{ Hz})$$

$$v[n] = |\vec{a}[n] - \vec{b}[n]| = \sqrt{\sum_k (a_k[n] - b_k[n])^2} \quad [g] \quad (6.2)$$

$\vec{b}$ is the gravity vector (plus any slow tilt) that the filter tracks; v is the size of the vibration, without gravity, in any direction.

Why per axis, not on the magnitude of the acceleration? The old version first computed the magnitude $|\vec{a}|$ and then subtracted the baseline. For a small vibration $\vec{\delta}$ on top of gravity, $|\vec{g} + \vec{\delta}| \approx g + \vec{\delta} \cdot \hat{g}$: the magnitude sees only the vertical component, and a sideways tap on the table all but vanishes. Subtracting the baseline on each axis shows the vibration in any direction. Every 50 Hz sample is processed by its own timestamp, not just the latest one received.

6.1.2 The shaking index

Shaking index — a relative logarithmic scale

$$\text{index} = \log_{10}(PGA_{\text{gal}}) \quad (6.3)$$

$$PGA_{\text{gal}} = v_{\text{peak}} \times 980.665 \text{ [cm/s}^2\text{, Gal]}.$$

Orders of magnitude: tapping the table $\rightarrow 0.5 \dots 1.5$; a strong impact $\rightarrow 2 \dots 3$.

This is not a magnitude. The formula has no distance correction, and the distance correction is exactly what turns a measured amplitude into a magnitude. What it measures is the shaking at the sensor — a quantity akin to intensity. The card used to be called **Richter** (ML est.); it is now **Shaking index**.

6.1.3 Chart scale and status thresholds

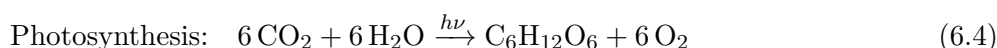
The seismogram has a fixed scale of $\pm 0.25 g$, so small shocks stay visible and two recordings can be compared by eye. The measured noise of a box at rest, after the filter, is about $\pm 0.01 g$;

the CALM threshold sits just above it.

Peak v [g]	Status
< 0.012	CALM (within the sensor noise)
0.012...0.08	Vibration (light → moderate)
> 0.08	Impact / shock

6.2 CO₂ and Photosynthesis

6.2.1 Chemical context



6.2.2 Independent rates of change

Separate timers for CO₂ and LUX

$$\dot{C}_{\text{CO}_2} = \frac{C_2 - C_1}{(t_2 - t_1)/60} \quad [\text{ppm/min}] \quad (6.6)$$

$$\dot{E}_{\text{LUX}} = \frac{E_2 - E_1}{(t_2 - t_1)/60} \quad [\text{lx/min}] \quad (6.7)$$

Each has its own 30 s timer (`_prev_co2_time` and `_prev_lux_time`) — a shared timer would mean the LUX rate never updates. For the value you report, use the regression slope over at least 3–5 minutes (see the SCD41 section).

6.2.3 Net and gross photosynthesis

In the light, the jar shows *net photosynthesis*: CO₂ falls by the difference between what the plant fixes and what it respire ($R_{\text{light}} < 0$). In the dark only respiration shows ($R_{\text{dark}} > 0$). Respiration continues during the day too, so:

$$R_{\text{gross}} = |R_{\text{light}}| + |R_{\text{dark}}| \quad (6.8)$$

is the *gross* photosynthesis rate. Measure both rates in the same jar, with the same plant, and run a control (a jar with soil, no plant) — moist soil respire too.

6.2.4 Molar CO₂ flux (advanced calculation)

Mass flux from the concentration change (ideal gas)

$$\dot{m}_{\text{CO}_2} [\text{g/min}] = \dot{C} [\text{ppm/min}] \times 10^{-6} \times \frac{P [\text{Pa}] \times V [\text{m}^3]}{R \times T [\text{K}]} \times 44.01 [\text{g/mol}] \quad (6.9)$$

$R = 8.314 \text{ J/(mol K)}$; V = the volume of air in the enclosed space, **in m³** (1 L = 10^{−3} m³); P in pascals (1 hPa = 100 Pa); 10^{−6} turns ppm into a mole fraction. The factor PV/RT is the total number of moles of air in the container.

Example: a 5 L jar ($V = 0.005 \text{ m}^3$), $P = 101\,325 \text{ Pa}$, $T = 293 \text{ K}$, $\dot{C} = -20 \text{ ppm/min}$:
 $n_{\text{air}} = \frac{101\,325 \times 0.005}{8.314 \times 293} \approx 0.208 \text{ mol}$; $\dot{m} \approx 20 \times 10^{-6} \times 0.208 \times 44.01 \approx 1.8 \times 10^{-4} \text{ g/min} = 0.18$

mg/min fixed.

(The previous edition asked for V in litres together with P in pascals, which gave a result 1000 times too large.)

Below 400 ppm the sensor leaves the range in which its accuracy is guaranteed, and hours spent there can influence ASC self-calibration (see the SCD41 section).

6.3 Pressure and Altitude

The barometric formula (Ch. 2) applied to the school building: going up, pressure falls by ≈ 0.12 hPa per metre, i.e. by ≈ 0.35 hPa per floor.

- The exercise table has 5 rows: the ground floor, the floors climbed, and **Ground floor (return)**. Returning to the ground floor shows how much the atmospheric pressure changed during the measurement (weather typically changes it by $0.1 \dots 0.3$ hPa/h); the difference can be subtracted from the other readings.
- On each floor, wait 1–2 minutes before reading: the sensor and the air in the box need time to settle, and the average steadies. The **Save the value** button fills the current row of the table.
- Pressure is shown with one decimal (two at **Scientist**), and the pressure chart spans at least 1 hPa, so noise does not look like a change.

Chapter 7

Explore apps

7.1 Air Quality — several boxes

Up to 8 boxes with an SCD41 can be connected, each in a different spot in the classroom. Each box has its own colour and name on every chart.

Alert logic:

$$\Delta\text{CO}_2 = \max_i(C_i) - \min_i(C_i)$$

$\Delta > 600 \text{ ppm} \Rightarrow$ urgent ventilation; $\Delta > 300 \Rightarrow$ significant difference

A useful order of magnitude for the classroom: 30 pupils in a room of about 170 m^3 , windows closed, raise CO_2 by roughly 1000 ppm in 30 minutes.

7.2 Heat Islands with Several Boxes

Same architecture, with the BME280: compares the temperature in the corners of the classroom, in the hallway and outdoors. Before comparing, keep the boxes side by side for a few minutes so the differences between the sensors are known. Outdoors, a box without a ventilated shelter, left in the sun, also measures the radiation heating it, not just the air: “sun versus shade” differences of several degrees are mostly radiation error. The urban heat island proper is the difference in *air* temperature between the town and its surroundings, and it is largest at night.

7.3 Data export (PNG and Excel)

- **PNG:** saves every chart of the current step.
- **Excel (openpyxl)** from an exercise step: a metadata header (title, step, date, box), then the *raw* (unfiltered) readings of that step, with the **elapsed_s** column. The “Journal” sheet keeps the predictions and observations the pupils typed.
- The pupils’ journal is also saved as a text file in the **logs** folder, named:
`<exercise>_<box>_<date>_journal.txt`.

Pedagogical rationale: an exercise stopped at step 2 of 6 still produces useful data — it can be compared with other classes or with future measurements.

Chapter 8

Windows: installation and troubleshooting

8.1 Installing the package

- **Where to unzip.** Unzip the package into **Documents** or onto a stick, **not** into **Program Files**: the application may not write there, and the logs would end up in the fallback folder `MagicBox\logs` in **Documents**.
- **SmartScreen.** On first start, Windows may show “Windows protected your PC”, because the application is not digitally signed. Click **More info** → **Run anyway**.
- **The manuals** are in `MagicBox\docs`, next to the application; each activity’s **Open guide** button opens the matching PDF.

8.2 USB and Wi-Fi connection

- **USB drivers.** ESP32 boards use a CP210x (Silicon Labs) or CH340 (WCH) USB-serial adapter. If a box connected by USB does not appear in the application, nor in *Device Manager* under *Ports (COM & LPT)*, install the adapter maker’s driver. Use a USB data cable, not a charge-only one.
- **Firewall.** On first start, Windows asks whether the application may communicate on the network. Tick **Private networks** and allow access: the boxes connect to the computer on **TCP 8765**, after the application announces itself by **UDP** broadcast (port 8766). The classroom Wi-Fi must be marked as *private* in Windows; on a *public* network the firewall blocks the connections.
- **A box does not appear over Wi-Fi.** Check, in order: the firewall (above); that box and computer are on the same network; that the network does not isolate clients or require WPA2-Enterprise authentication; as a fallback, a phone hotspot. A box that fails to connect retries by itself at most every 60 s.

8.3 Common problems

Symptom	Remedy
A box lying on the table shows $a_z \approx -1g$; the plane flies backwards	Firmware older than 2.2.1 / 1.3.0 — reflash the box (the application shows the warning). With current firmware: the sensor is mounted the other way — tick Motion sensor upside down for that box in Settings .
Light stops at ≈ 3800 lx in the sun	Firmware older than 2.2.5 / 1.3.4 — reflash the box.
No CO ₂ values	serial_only firmware older than 1.3.4 (does not read the SCD41) — reflash; or wait 30 s after start-up.
CO ₂ too low or too high in clean air	Forced recalibration outdoors (SCD41 section).
Temperature higher than the room	The box warms itself; compare boxes side by side.
Logs are not next to the application	The application folder is not writable; look in MagicBox\logs in Documents .
Excel opens the CSV in a single column	Export as .xlsx (the default) or use the CSV export with ;.

Appendix A

Quick reference — JSON formats

A.1 Multi-sensor packet (1 Hz)

```
{
  "id":    "BOX_A1B2",
  "ts_ms": 123456,
  "data": {
    "lux":    312.5,
    "white":  1840,
    "raw_als": 1821,
    "T":      22.34,
    "H":      58.12,
    "P":      1013.28,
    "ALT":    0.3
  }
}
```

A.2 IMU packet (50 Hz)

```
{
  "id":    "BOX_A1B2",
  "ts_ms": 123476,
  "imu_hz": 50,
  "imu":    [[-0.023, 0.014, 0.998, 0.152, -0.089, 0.041]]
}
```

A.3 SCD41 + BME280 packet

```
{
  "id":    "BOX_C3D4",
  "ts_ms": 5000,
  "data": {
    "co2":    842,
    "TEMP_SCD": 23.1,
    "HUM_SCD": 55.4,
    "T":      22.98,
    "H":      57.2,
    "P":      1011.45
  }
}
```

Appendix B

I²C wiring

All sensors share the same I²C bus. A common GND is mandatory.

Sensor	SDA	SCL	VCC
MPU-6500	GPIO 21	GPIO 22	3.3 V
BME280	GPIO 21	GPIO 22	3.3 V
VEML7700	GPIO 21	GPIO 22	3.3 V
SCD41	GPIO 21	GPIO 22	3.3 V

Sensor	I ² C address (7-bit)
MPU-6500	0x68 (AD0=GND) or 0x69 (AD0=VCC)
BME280	0x76 or 0x77 (SDO pin)
VEML7700	0x10 (fixed)
SCD41	0x62 (fixed, Sensirion)

Critical constraint: the SCD41 limits the bus to a maximum of 100 kHz. At 400 kHz, the MPU-6500 “disappears” from the bus due to the SCD41’s clock-stretch interference.